

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1. A hospital management system requires a SOAP-based web service to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a SOAP-based web service for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

ANSWER:

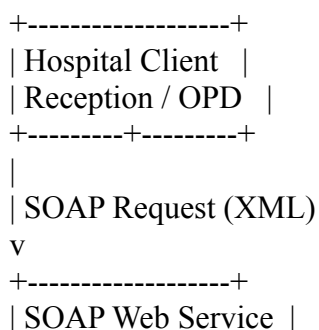
SOAP-Based Web Service for Hospital Management System

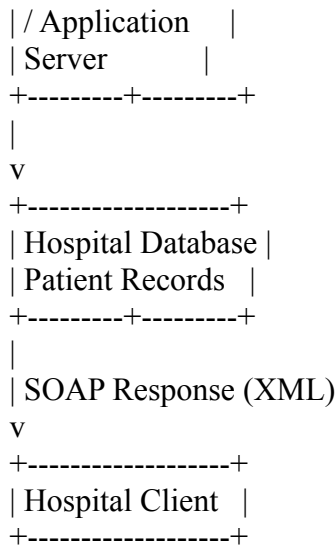
Introduction

A hospital management system can use a **SOAP (Simple Object Access Protocol) web service** to allow different hospital departments such as **reception, outpatient department (OPD), laboratory, pharmacy, billing, and medical records** to communicate with each other.

SOAP provides a **standard XML-based communication mechanism** between a client and a server. Since hospital applications exchange sensitive patient information, SOAP is suitable because it supports **structured messaging, security standards, reliability, and interoperability**.

Basic Architecture





SOAP Message Structure

A SOAP message is an **XML document** consisting mainly of:

Envelope

Header

Body

General Structure

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

```
<soap:Header>
<!-- Security / authentication information -->
</soap:Header>
```

```
<soap:Body>
<!-- Request or response information -->
</soap:Body>
</soap:Envelope>
```

1. Envelope

The **Envelope** is the root element of every SOAP message.
It tells the receiver that the XML document is a SOAP message.

```
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

It contains the **Header** and **Body**.

2. Header

The **Header** is optional and contains information that is not directly related to the main business data.
For a hospital system, it can contain:

Authentication information

Authorization information

Security tokens

Transaction information

Message identification

Routing information

Digital signatures

Example:

```
<soap:Header>
<Security>
<Username>hospitalUser</Username>
<Token>ABC123</Token>
```

</Security></soap:Header>

For a real production system, sensitive credentials should **not** be sent as plain text like this; standards such as **WS-Security** can be used for secure SOAP messaging.

3. Body

The **Body** contains the actual request or response.

For example, a request to add a patient could contain:

<soap:Body>

<AddPatient>

<PatientID>P1001</PatientID>

<Name>Ravi Kumar</Name>

<Age>35</Age>

<Gender>Male</Gender>

<Contact>9876543210</Contact>

</AddPatient></soap:Body>

The server processes this information and sends a SOAP response.

Operations of the Hospital Web Service

The web service can provide several operations.

Operation	Purpose
addPatient()	Registers a new patient
getPatient()	Retrieves patient information
updatePatient()	Updates patient information
deletePatient()	Removes a patient record where permitted
scheduleAppointment()	Creates an appointment
getAppointments()	Retrieves appointments
addMedicalRecord()	Stores medical information
getMedicalRecords()	Retrieves medical records
cancelAppointment()	Cancels an appointment

Example: Add Patient

The client sends:

<soap:Envelope

xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

<soap:Header>

<Security>

<Token>ABC123</Token>

</Security>

</soap:Header>

<soap:Body>

<AddPatient>

<PatientID>P1001</PatientID>

<Name>Ravi Kumar</Name>

<Age>35</Age>

<Gender>Male</Gender>

</AddPatient>

</soap:Body>

</soap:Envelope>

The server validates the request, stores the patient information in the database, and returns:

<soap:Envelope

```
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
```

```
<soap:Body>  
<AddPatientResponse>  
<Status>Success</Status>  
<Message>Patient registered successfully</Message>  
<PatientID>P1001</PatientID>  
</AddPatientResponse>  
</soap:Body>  
</soap:Envelope>
```

Example: Get Medical Records

The client sends:

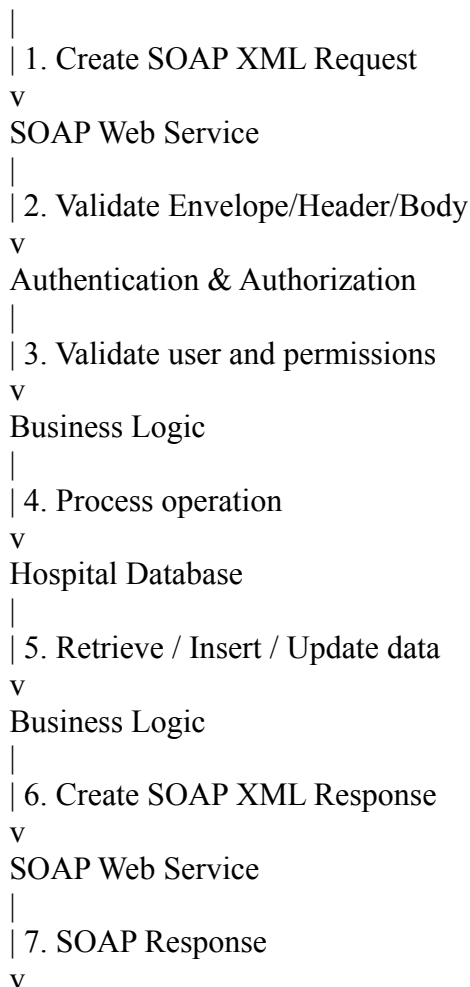
```
<soap:Body>  
<GetMedicalRecords>  
<PatientID>P1001</PatientID>  
</GetMedicalRecords></soap:Body>
```

The server retrieves the information from the database and returns:

```
<soap:Body>  
<GetMedicalRecordsResponse>  
<PatientID>P1001</PatientID>  
<Diagnosis>Fever</Diagnosis>  
<Doctor>Dr. Kumar</Doctor>  
<Prescription>Paracetamol</Prescription>  
</GetMedicalRecordsResponse></soap:Body>
```

Data Exchange Between Client and Server

Client



Client

Why SOAP is Suitable for a Hospital System

1. Security

SOAP supports standards such as **WS-Security**, which can provide:

- Authentication
- Message integrity
- Digital signatures
- Encryption
- Security tokens

This is important because medical information is confidential.

2. Reliability

SOAP-based web services can use specifications such as **WS-ReliableMessaging** where required to support reliable message delivery and ordering.

3. Interoperability

Different hospital departments may use different programming languages and operating systems. SOAP uses standardized XML messaging, allowing systems implemented in different technologies to communicate.

4. Structured Communication

SOAP has a well-defined message structure consisting of **Envelope, Header, and Body**.

5. Error Handling

SOAP provides a standardized **SOAP Fault** mechanism for reporting errors.

6. Enterprise Support

SOAP has a mature ecosystem of WS-* standards for security, reliability, transactions, and related enterprise requirements.

Conclusion: SOAP is appropriate when the hospital requires **strict contracts, strong enterprise security, reliable messaging, and interoperability** between multiple systems.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use WSDL (Web Services Description Language) to describe the service.

Design the WSDL structure for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

ANSWER:

WSDL Structure for an Enterprise Web Service Introduction

WSDL (Web Services Description Language) is an XML-based language used to describe a web service.

It tells a client:

What operations are available

What input is required
What output is returned
What data types are used
How messages are formatted
Which protocol is used
Where the service is available

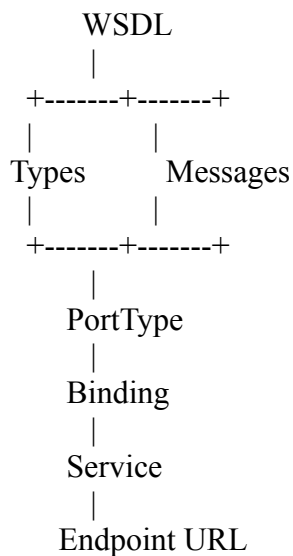
Therefore, WSDL acts like a **contract between the service provider and service consumer**.

Main Components of WSDL

The important components are:

1. **Types**
2. **Message**
3. **PortType**
4. **Binding**
5. **Service**

The relationship can be represented as:



1. Types

The <types> section defines the **data types** exchanged between the client and server.

XML Schema (XSD) is commonly used.

Example:

```
<types>
  <xsd:schema
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">

    <xsd:element name="GetPatientRequest">
      <xsd:complexType>
        <xsd:sequence>
          <xsd:element name="PatientID"
```

```

        type="xsd:string"/>
    </xsd:sequence>
</xsd:complexType>
</xsd:element>

</xsd:schema></types>

```

Here, PatientID is defined as a string.

2. Message

The <message> element defines the **logical messages exchanged** between the client and service.

For example:

```

<message name="GetDataRequest">
    <part name="patientID" type="xsd:string"/></message>
<message name="GetDataResponse">
    <part name="patientName" type="xsd:string"/>
    <part name="status" type="xsd:string"/></message>

```

The request contains patient ID, while the response contains patient name and status.

3. PortType

The <portType> defines the **operations provided by the web service**.

It is similar to an interface in programming.

Example:

```

<portType name="HospitalPortType">

    <operation name="GetData">
        <input message="tns:GetDataRequest"/>
        <output message="tns:GetDataResponse"/>
    </operation>

    <operation name="UpdateData">
        <input message="tns:UpdateDataRequest"/>
        <output message="tns:UpdateDataResponse"/>
    </operation>
</portType>

```

This tells the client that the service provides:

```

    GetData
    UpdateData

```

4. Binding

The <binding> element specifies **how the operations are communicated**.

It defines details such as:

- Communication protocol
- SOAP version
- Message style
- Encoding
- Operation details

Example:

```
<binding name="HospitalBinding"
  type="tns:HospitalPortType">

  <soap:binding
    transport="http://schemas.xmlsoap.org/soap/http"
    style="document"/>

  <operation name="GetData">
    <soap:operation soapAction="GetData"/>
  </operation>
</binding>
```

Here, the service uses **SOAP over HTTP**.

5. Service

The <service> element specifies the actual service endpoint.

Example:

```
<service name="HospitalService">

  <port name="HospitalPort"
    binding="tns:HospitalBinding">

    <soap:address
      location="https://example.com/HospitalService"/>

  </port>
</service>
```

The soap:address provides the location where the service can be accessed.

Complete WSDL Structure

A simplified WSDL document can be represented as:

```
<definitions
  name="HospitalService"
  targetNamespace="http://example.com/hospital"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```

xmlns:xsd="http://www.w3.org/2001/XMLSchema">

<!-- 1. Data Types -->
<types>
  ...
</types>

<!-- 2. Request and Response Messages -->
<message name="GetDataRequest">
  ...
</message>

<message name="GetDataResponse">
  ...
</message>

<!-- 3. Operations -->
<portType name="HospitalPortType">
  <operation name="GetData">
    <input message="tns:GetDataRequest"/>
    <output message="tns:GetDataResponse"/>
  </operation>
</portType>

<!-- 4. Communication Details -->
<binding name="HospitalBinding"
  type="tns:HospitalPortType">
  ...
</binding>

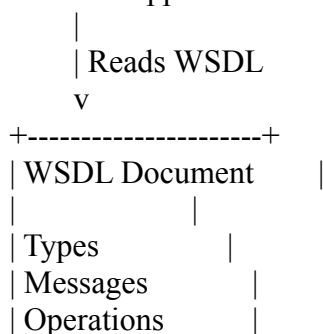
<!-- 5. Service Endpoint -->
<service name="HospitalService">
  <port name="HospitalPort"
    binding="tns:HospitalBinding">
    <soap:address
      location="https://example.com/HospitalService"/>
    </port>
  </service>
</definitions>

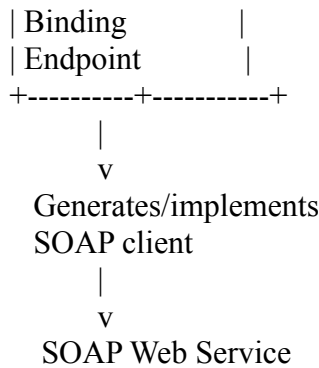
```

How WSDL Helps Service Integration

Consider an external company that wants to communicate with the hospital service.

External Application





The client does not have to manually guess:

- What operation names are available
- What parameters are required
- What response will be returned
- Which protocol should be used
- Where the service is located

The WSDL provides this information in a machine-readable form.

Importance of WSDL

Standardized Interface

WSDL provides a standard description of the service.

Platform Independence

A Java application can communicate with a .NET or other SOAP implementation when they follow the same service contract.

Automatic Client Generation

Many development tools can generate client-side code from WSDL.

Reduced Integration Errors

The client knows the expected data types, operations, and message structures.

Loose Coupling

The service consumer interacts with the defined interface rather than depending on the internal implementation.

Conclusion: WSDL is important because it provides a **formal, machine-readable contract** that allows different applications and organizations to integrate with a web service consistently.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

ANSWER:

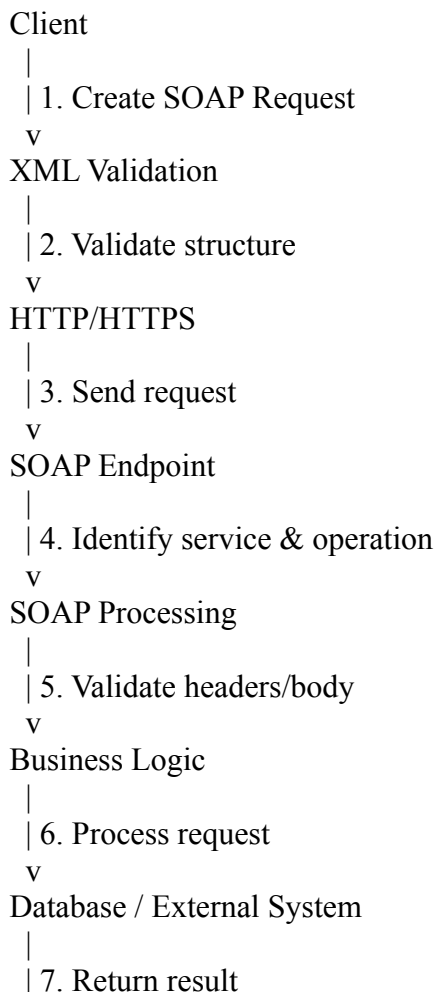
Debugging Problems in SOAP Communication Introduction

Suppose a company has a SOAP-based system where:

Requests are not processed.
Responses are delayed.
Incorrect responses are received.
Some clients can communicate while others cannot.

These problems can occur at different levels, including **XML formatting, namespaces, SOAP headers, endpoint configuration, authentication, server processing, network connectivity, and database/business logic.**

SOAP Communication Process



v
SOAP Response
|
| 8. Send response
v
Client

Possible Problems

1. Incorrect SOAP Message Formatting

SOAP requires a specific XML structure.

Incorrect:

```
<soap:Envelope>  
  <soap:Body>  
    <GetData>  
  </soap:Body></soap:Envelope>
```

The XML is malformed because the GetData element is not properly closed.

Correct:

```
<soap:Envelope>  
  <soap:Body>  
    <GetData>  
    </GetData>  
  </soap:Body></soap:Envelope>
```

Solution

Validate the XML using:

- XML validators
- IDE XML validation
- SOAP testing tools
- Schema validation

2. Incorrect Namespace

Namespaces are extremely important in SOAP.

For example, the server expects:

```
xmlns:ns="http://example.com/hospital"
```

but the client sends a different namespace:

```
xmlns:ns="http://example.com/patient"
```

The server may fail to recognize the operation even though the element names appear correct.

Solution

Check:

- Namespace URI
- Prefixes
- Target namespace
- WSDL definitions
- XML Schema definitions

Remember that **the namespace URI matters; the prefix itself is only an alias.**

3. Incorrect Endpoint

The client may send the request to an incorrect URL.

For example:

Client
|
| Request
v
https://server.com/wrongEndpoint

The server may return:

- HTTP 404
- Connection error
- Timeout
- Service unavailable

Solution

Verify the endpoint from the WSDL and configuration.

4. Wrong SOAP Version

A client may use SOAP 1.1 while the service expects SOAP 1.2, or vice versa.

SOAP 1.1 commonly uses:

Content-Type: text/xml

SOAP 1.2 commonly uses:

Content-Type: application/soap+xml

Solution

Check:

- WSDL
- HTTP headers
- SOAP namespace
- Server configuration

Client configuration

5. Incorrect SOAPAction

Some SOAP 1.1 services use the HTTP SOAPAction header to identify the requested operation.

Example:

SOAPAction: "GetPatient"

If the service expects a different action, the request may not be routed correctly.

Solution

Compare the SOAPAction with the WSDL and server configuration.

6. Authentication and Authorization Problems

The request may be rejected because:

- Authentication token is missing
- Token is expired
- User is not authorized
- Security header is incorrect
- Certificate is invalid

Solution

Check the SOAP Header and security configuration.

7. Server-Side Business Logic Errors

The SOAP message may be completely correct, but the application itself may have an error.

For example:

SOAP Request

|

v

SOAP Processing ---- Correct

|

v

Business Logic ----- Error

|

v

Database ----- Failure

Solution

Check:

Application logs

- Database logs
- Exception logs
- Service implementation
- SQL queries
- Input validation

8. Network and Timeout Problems

A response may be delayed because:

- Server is overloaded
- Network connection is slow
- Database query takes too long
- Firewall blocks traffic
- Proxy configuration is incorrect
- Service is unavailable

Solution

Check:

- Network connectivity
- DNS
- Firewall
- Proxy
- Server CPU/memory
- Database performance
- Request timeout settings

9. Incorrect Response Mapping

Sometimes the server returns a response, but the client interprets it incorrectly.

For example, the server returns:

```
<GetDataResponse>  
  <PatientName>Ravi</PatientName></GetDataResponse>
```

while the client expects:

```
<GetDataResponse>  
  <Name>Ravi</Name></GetDataResponse>
```

This can cause deserialization or mapping problems.

Solution

Compare the response against:

- WSDL
- XSD
- Generated client classes

Expected response schema

SOAP Fault Handling

SOAP provides a standard mechanism called **SOAP Fault** for reporting errors.

Example:

```
<soap:Fault>  
  <faultcode>soap:Server</faultcode>  
  <faultstring>Unable to retrieve patient record</faultstring></soap:Fault>
```

A more modern SOAP fault structure can include fields such as Code, Reason, and Detail depending on the SOAP version.

Common Fault Categories

Client-side error

soap:Client

The request itself is invalid.

Examples:

- Invalid XML
- Missing required parameter
- Invalid data

Server-side error

soap:Server

The server could not process a valid request.

Examples:

- Database failure
- Internal application error
- Service failure

Systematic Debugging Approach

A good debugging process should proceed from the outside toward the application logic.

Step 1: Check Endpoint

Verify that the client is communicating with the correct service URL.

Step 2: Check Network

Test:

DNS
Port
Firewall
HTTPS certificate
Proxy

Step 3: Capture the SOAP Request

Inspect the complete request.

Check:

Envelope
Header
Body
Namespace
Operation
Parameters

Step 4: Validate XML

Confirm that the request is well-formed XML.

Step 5: Validate Against WSDL/XSD

Check whether the message follows the service contract.

Step 6: Check SOAP Version

Ensure client and server use compatible SOAP versions.

Step 7: Check Headers

Verify:

Content-Type
SOAPAction where applicable
Authentication headers
WS-Security information

Step 8: Check Server Logs

Look for:

Parsing errors
Authentication failures
Application exceptions
Database errors

Step 9: Check Database

Verify that:

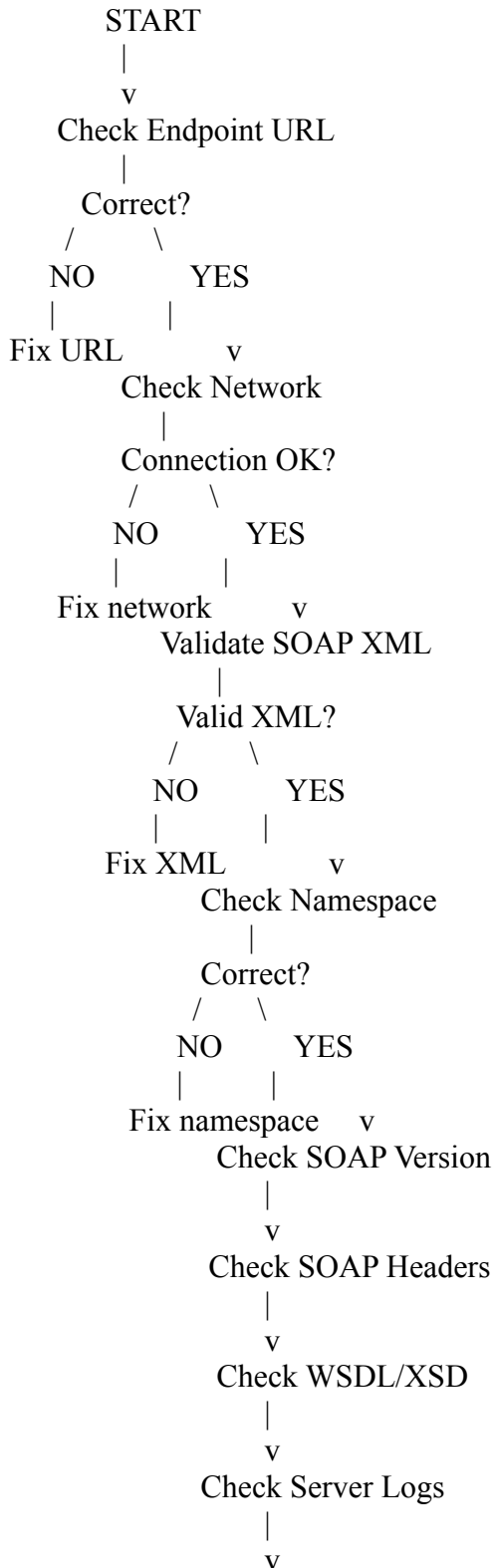
Required data exists

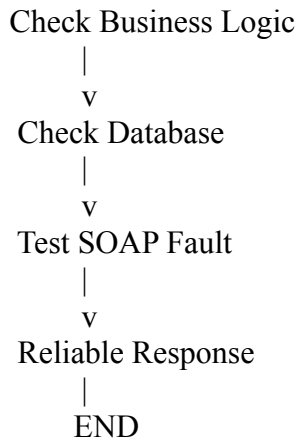
Queries execute correctly
Database connections work
Transactions complete correctly

Step 10: Test SOAP Fault Handling

Send an intentionally invalid request and verify that the service returns a meaningful SOAP Fault instead of hanging or returning an ambiguous response.

Debugging Flowchart





How the Corrected System Ensures Reliable Communication

After debugging, the system should implement the following practices:

1. WSDL-Based Contract Validation

The client and server should follow the same WSDL and schema definitions.

2. XML Validation

Every request and response should conform to the expected XML structure.

3. Correct Namespace Management

The namespace URIs must exactly match the service contract.

4. Proper Endpoint Configuration

Clients must use the correct service endpoint.

5. Secure Communication

Use **HTTPS/TLS** for transport security and appropriate **WS-Security** mechanisms where message-level security is required.

6. Proper Error Handling

Use SOAP Faults to clearly communicate processing failures.

7. Logging and Monitoring

Maintain logs for:

- Requests
- Responses
- Exceptions
- Authentication failures
- Performance
- Server errors

Sensitive patient information should not be unnecessarily written to logs.

8. Timeout and Retry Strategy

Transient failures can be handled using carefully designed timeout and retry policies. Retries should be used safely because repeating some operations can create duplicate effects.

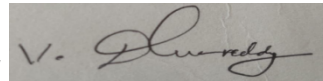
9. Transaction and Reliability Mechanisms

For enterprise systems that require guaranteed message delivery or ordering, appropriate WS-* specifications such as **WS-ReliableMessaging** can be considered.

Code of Conduct:

I __ V . D h a r a n i r e d d y (1 9 2 3 1 1 2 1 9) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results	Good analysis with reasonable	Basic analysis with limited	Poor or no

		and performance evaluation	validation	validation	analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation